Borealis

外部の計算コードの入力を探す、逆問題と最適化の枠組み

仕組み・手法・並列化・検証

高橋裕介
北海道大学

2026 年 10 月 20 日 (Borealis version 1.7.0)

概要　Borealis は、外部の計算コード（または簡単な関数）の出力が基準のデータに近づくように、その入力のパラメータを探す Python の枠組みである。最適化の手法（粒子群最適化 PSO、人工蜂コロニー ABC、遺伝的アルゴリズム GA、ベイズ最適化）と、対象とする計算コード（軌道解析の Tacode、熱伝導、SU2 と preCICE による形状最適化など）を「アダプタ」として切り離し、YAML の設定ファイル 1 本で組み合わせを切り替える。PSO・ABC・GA は mpi4py で候補を分担して評価でき、PSO と GA は多目的の最適化もできる。外部コード自体が MPI で並列に走る場合のために、Borealis を 1 プロセスで起動して候補をスレッドで同時に評価する並列評価も入っている。本稿は、このコードが何をどう計算しているか、どう並列化しているか、そして解析解の分かっている問題でどこまで答えに近づくかを述べる。挙げる数値は、断りのない限りリポジトリの検証ケース（`report/verification/`）と測定（`report/data/`）が出したもので、図もその出力から作っている。

目次

1 このレポートの範囲 1
2 問題の設定 1
3 コードの構成 2
4 最適化の手法 3
5 並列化 6
6 アダプタ 7
7 出力 8
8 検証 8
9 適用の例：ISS の軌道からの大気密度の推定 12
10 既知の問題 12

1　このレポートの範囲

Borealis が解くのは、次の形の問題である。計算コードがあり、入力のパラメータ $\mathbf{x}$ を与えると出力を返す。観測や別の計算から得た基準のデータがあり、出力をそれに近づけたい。そこで、出力と基準のデータの差を目的関数 $f(\mathbf{x})$ とし、それを小さくする $\mathbf{x}$ を探す。計算コードの中身には立ち入らず、入力を書き換えて走らせ、出力を読むことだけを繰り返す。いわゆるブラックボックスの最適化である。

このレポートには、コードが何を仮定しているか（節 2）、どう組み立てられているか（節 3）、4 つの手法が

何を計算しているか（節 4）、どう並列化しているか（節 5）、そして答えが解析解からどれだけ離れているか（節 8）を書く。手法の式は、文献の一般的な形ではなく、**ソースに書いてあるとおりの形**で書く。文献の標準の形と違うところは、その場でそう断る。

`README.md` は概要である。設定ファイルのキーは、すべて `report/borealis_config_ja.pdf` に並べてある。まだ直っていないこと・確かめていないことは、リポジトリの外に置いた開発の記録に番号つきでまとめてあり、本文では「既知の問題 7」のように番号で引く。

2 問題の設定

2.1 パラメータと範囲

パラメータは、設定ファイルの `parameter_optimized.boundary` に、名前（`name`）と成分（`component`）の入れ子で書く。成分ごとに名前（`type`）と下限・上限（`bound_min`・`bound_max`）を持ち、すべての成分を順に並べたものがパラメータのベクトル $\mathbf{x} = (x_1, \ldots, x_d)$ になる。`name` は、外部コードの制御ファイルの中で書き換える変数を探す手がかりに使う（使い方はアダプタごとに違う）。

範囲は、手法ごとに次のように効く。

- PSO・ABC・GA は初期値を、ベイズ最適化は初期の点を、この範囲の一様乱数から取る。ベイズ最適化は、獲得関数を最大にする点もこの範囲の中で探す
- ABC と GA は、新しい候補をこの範囲の中に収める
- **PSO は切り詰めない**。粒子は範囲の外へ出うる。そのとき外部コードには範囲の外の値が渡る（既知の問題 21）

2.2 目的関数とペナルティ

目的関数は、アダプタの `objective_function` が返す実数（多目的なら実数の並び）である。外部コードを呼ぶアダプタでは、たいてい計算の出力と基準のデータの差の二乗の和か、その平均の平方根である。

`flag_penalty: True` のとき、アダプタはパラメータが範囲の外にあれば `penalty_value` を足す。

$$\tilde{f}(\mathbf{x}) = f(\mathbf{x}) + P(\mathbf{x}), \qquad P(\mathbf{x}) = \begin{cases} p & \text{ある } i \text{ で } x_i \notin [x_i^{\min}, x_i^{\max}] \\ 0 & \text{それ以外} \end{cases} \tag{1}$$

p は違反の個数にも距離にもよらない定数である（既知の問題 13）。範囲の外は平らな面になるので、ペナルティだけでは粒子を範囲の中へ引き戻す勾配は生まれない。

2.3 最小化と最大化

4 つの手法は、どれも最小化として動く。`maximize: True` のとき、PSO・ABC・GA は符号 $s = -1$ を目的関数の引数として渡し、**符号を掛けるのはアダプタの側**である（ペナルティは符号を掛けたあとに足す）。符号を掛けるアダプタは `flag_apply_sign` を宣言しており、いまは `shapeoptimizer`・`simple_function` である。ほかのアダプタは誤差を返すので最大化に意味が無く、`maximize: True` を書くと起動時に止まる（1.6 までは黙って最小化を続けていた。既知の問題 1）。ベイズ最適化は符号を自分で扱うので、どのアダプタでも最大化できる。

2.4 評価の失敗

外部コードが失敗したとき（終了ステータスが 0 でない、結果のファイルが書かれない、基準のデータと比べられる点が 1 つも無い）、アダプタは無限大を返す。PSO・ABC・GA は、目的関数の値に NaN・無限大があれば、警告を出して `parameter_optimized.value_nonfinite`（既定 10^{30}）に置き換える。1.6 までは NaN が `np.argmin` でリーダーを乗っ取り、失敗した外部コードの古い結果から誤差を計算しえた（既知の問題 2、7）。

3 コードの構成

3.1 3 つの層

- `src/borealis.py`：入口。設定ファイルの `adapter.kind_adapter` と `optimizer.kind_optimizer` の語を見て、対応するクラスを読み込んで組み合わせる
- `src/optimizer_*/`・`src/Bayesian_optimization/`：最適化の手法。アダプタの `objective_function` を呼ぶ側
- `src/adapter_*/`：計算コードを走らせ、目的関数の値を返す側。`src/adapter_template/` に雛形が 2 つある（解析的な関数だけの最小の形と、外部コードを呼ぶ完全な形）

アダプタとオプティマイザは、どちらも共通の基底のクラス `orbital`（`src/orbital/orbital.py`）を継ぐ。ディレクトリの操作、YAML の読み込み、ペナルティの計算、時刻の単位の換算などはここにある。MPI は `src/mpi/mpi.py` の薄い包みで、プロセスが 1 つなら逐次の動きになる。

3.2 呼ばれる順序

`borealis.py` は、アダプタとオプティマイザを次の順に呼ぶ。

1. `adapter.initial_settings(config)`：作業ディレクトリを作り、テンプレートのケースを写す。**作業ディレクトリは中身ごと消してから作る**
2. `adapter.reference_data_setting(config)`：基準のデータを読む
3. `optimizer.initial_setting(config)`：出力のディレクトリを作る（これも消してから作る）
4. `optimizer.boundary_setting(config)`：範囲を手法の形に直す（PSO・ABC・GA は $(x^{\min}, x^{\max})$ の組の並び、ベイズ最適化は通し番号の名前から範囲への辞書）
5. `optimizer.drive_optimization(config, adapter.objective_function, ...)`：最適化を回し、結果を書き出す

目的関数は `objective_function(x, id, step, sign)` の形で呼ばれる。`x` は $(1, d)$ の形の配列である（以前のベイズ最適化のライブラリ GPyOpt の呼び方に合わせたもの）。4 つの手法とも、評価ごとに一意な番号 `id`、ステップ `step`、符号 `sign` を渡す。アダプタは `id` をケースのディレクトリの番号に使う。

3.3 外部コードを呼ぶ流れ

外部コードを呼ぶアダプタの `objective_function` は、おおむね次の流れである。

1. テンプレートのケースを、**<work_dir>/<case_dir><番号>** に写す
2. 制御ファイルの中で、パラメータの名前の変数を探して値を書き換える
3. 外部コードを起動し、終わるのを待つ（`subprocess.call`）

4. 結果のファイルを読み、基準のデータと比べて誤差を計算する。範囲の外ならペナルティを足す

反復ごとのケースはすべて残るので、あとから最良の解のケースを取り出して調べられる（`src_helper/pickup_optimal_result`）。外部コードの起動は `orbital.run_external_code` にまとめてあり、起動の前にテンプレートから写した古い結果のファイルを消し、終了ステータスと結果のファイルを確かめる。子のプロセスには MPI の環境変数を渡さない（節 5.2）。

4 最適化の手法

以下、N を候補の数（粒子・雇用蜂）、d を次元、n をステップ、$\mathcal{U}(a,b)$ を一様乱数とする。乱数は numpy の大域の乱数で、`optimizer.seed` を書くと各ランクを $seed + rank$ で初期化する。

4.1 粒子群最適化 (PSO)

`src/optimizer_pso/optimizer_pso.py`。慣性の重みのついた PSO [1, 2] に、多目的のための非劣解のアーカイブ [3] を足したものである。

初期化。 粒子 i の位置 $\mathbf{x}_i$ は範囲の一様乱数から取る（`boundary_initial` があれば、すべての次元にその 1 つの範囲を使う）。速度 $\mathbf{v}_i$ は、`velocity_initial` があればその範囲、無ければ**パラメータの尺度によらず** $\mathcal{U}(-1,1)$ から取る。個人の最良の値 $f(\mathbf{p}_i)$ は $+\infty$ で始める。

1 **ステップ。** 各ステップで次を行う。

1. すべての粒子の現在の位置 $\mathbf{x}_i^n$ で目的関数を評価する（NaN・無限大は `value_nonfinite` に置き換える）
2. 新しい値が個人の最良を支配すれば（単目的なら厳密に小さければ）、個人の最良 $\mathbf{p}_i$ を更新する
3. **このステップの粒子の現在の位置だけから**アーカイブを作る。単目的なら最良の 1 点、多目的なら互いに支配されない点の集合である
4. アーカイブからリーダー $\mathbf{g}$ を選ぶ。単目的なら唯一の点、多目的なら一様に乱択する
5. 速度と位置を更新する

$$\mathbf{v}_i^{n+1} = w\,\mathbf{v}_i^n + c_1\,\mathbf{r}_1 \circ (\mathbf{p}_i - \mathbf{x}_i^n) + c_2\,\mathbf{r}_2 \circ (\mathbf{g} - \mathbf{x}_i^n), \tag{2}$$

$$\mathbf{x}_i^{n+1} = \mathbf{x}_i^n + \mathbf{v}_i^{n+1} \tag{3}$$

ここで w は `inertia`、c_1 は `cognitive_coef`、c_2 は `social_coef`、$\mathbf{r}_1, \mathbf{r}_2$ は成分ごとに独立な $\mathcal{U}(0,1)$ のベクトル、$\circ$ は成分ごとの積である。位置の切り詰めも、速度の上限も無い。

標準の PSO との違いは、リーダーである。標準の PSO の大域最良は「それまでに見つかった最良の点」だが、Borealis のリーダーは**そのステップの粒子の中の最良**で、過去のよい点は次のステップで消える（ソースのコメントにある意図した選択。既知の問題 10）。個人の最良は過去を覚えているので、群れが過去の最良を忘れるわけではない。多目的のリーダーを混雑距離で選ぶ関数（`select_global_best_cd`）も書いてあるが、使っていない。

収束の判定。 個人の最良の値の履歴から、各粒子の相対変化を

$$r_i^n = \frac{\left\| f(\mathbf{p}_i)^{(n)} - f(\mathbf{p}_i)^{(n-W+1)} \right\|}{\left\| f(\mathbf{p}_i)^{(0)} \right\| + 10^{-15}} \tag{4}$$

で計算し、粒子の平均 $\bar{r}^n$ を「群れの残差」とする。W は `windowsize_residual`（既定 20）、$f(\mathbf{p}_i)^{(0)}$ は

最初のステップのあとの個人の最良の値である。$\bar{r}^n \le$ `tolerance_residual`（既定 10^{-5}）で、かつ $\bar{r}^n$ が初めて計算した残差と同じでないとき（W ステップのあいだ何も変わらなかった、という見かけの収束を除くため）、単目的なら収束とする。

多目的では、それに加えてアーカイブの停滞を見る。アーカイブの点の値の平均（重心）$\bar{\mathbf{f}}^n$ を取り、$\|\bar{\mathbf{f}}^n - \bar{\mathbf{f}}^{n-W_a+1}\| \le$ `tolerance_archive`（既定 1.0、W_a は `windowsize_archive`、既定 5）のとき停滞とし、残差と停滞の両方がそろったら収束とする。

評価の回数。 1 ステップに N 回である。

4.2 人工蜂コロニー (ABC)

`src/optimizer_abc/optimizer_abc.py`。Karaboga の ABC [4, 5] である。単目的だけを扱う（多目的の設定を拒まないのは既知の問題 12）。

N 個の食料源 $\mathbf{x}_i$ を範囲の一様乱数から作り、評価する。各ステップは 3 つの段からなる。

雇用蜂。 各 i について、次元 k を 1 つと、$j \ne i$ を 1 つ乱択し、

$$v_k = x_{ik} + \phi\,(x_{ik} - x_{jk}), \qquad \phi \sim \mathcal{U}(-1, 1) \tag{5}$$

とする（ほかの次元は $\mathbf{x}_i$ のまま）。v_k は範囲に切り詰める。適応度

$$\mathrm{fit}(f) = \begin{cases} 1/(1+f) & f \ge 0 \\ 1 + |f| & f < 0 \end{cases} \tag{6}$$

が上がれば $\mathbf{x}_i$ を $\mathbf{v}$ に置き換えて訪問の回数を 0 に戻し、上がらなければ回数を 1 足す。

追従蜂。 M 匹（`num_onlook_bee`）が、確率 $\mathrm{fit}_i / \sum_l \mathrm{fit}_l$ で食料源 i を選び（ルーレット選択）、雇用蜂と同じ近傍の探索をする。

偵察蜂。 訪問の回数が `visit_limit`（古い綴り `vist_limit` も読む）を**超えた**食料源は捨て、範囲の一様乱数から作り直して評価する。**最良の食料源もこの対象になる。**

最良と収束。 ABC は、それまでの最良を食料源とは別に持つ。偵察蜂が最良の食料源を捨てても、それまでの最良は失われない。出力のファイル `output_global.dat` には、各ステップの時点でのそれまでの最良を書く（1.6 までは各ステップの食料源の中の最良を書いていて、悪化しえた。既知の問題 16、節 8.1）。収束は、W を `windowsize_residual`（既定 1）として $\mathrm{mean}_i |f_i^n - f_i^{n-W}| / (|f_i^0| + 10^{-20}) \le$ `tolerance` で判定し、最初の W ステップは判定しない。改善の無いステップではこの値が 0 になるので、早く止まりすぎるなら W を大きくする（既知の問題 5）。

評価の回数。 1 ステップに $N + M$ 回と、偵察蜂の数である。評価の番号は 10000× ステップ +1000× 段 + 蜂の番号 で作るので、N や M が 1000 以上だと番号が重なる。

4.3 遺伝的アルゴリズム (GA)

`src/optimizer_ga/optimizer_ga.py`。実数値の GA を、NSGA-II [9] の世代交代で回す。単目的では、これは最良の個体が必ず残る $(\mu + \lambda)$ のエリート保存の GA になり、2 目的以上ではそのまま NSGA-II になる。

1 世代。 N 個の親から N 個の子を作って評価し、親と子を合わせた $2N$ 個から次の N 個を残す。1 世代に評価するのは子の N 個だけで、親の値は持ち越す。

1. **選択**：`tournament_size`（既定 2）個を乱択し、非劣解の順位の小さいもの、同じなら混雑距離の大きいものを親とする
2. **交叉**：確率 `crossover_rate`（既定 0.9）で SBX [7] を行う。成分ごとに確率 0.5 で、親 $y_1 < y_2$ から

$$c_{1,2} = \tfrac{1}{2}\left[(y_1 + y_2) \mp \beta_q (y_2 - y_1)\right] \tag{7}$$

   を作る。β_q は分布指数 η_c（`eta_crossover`、既定 15）の多項式の分布から、子が範囲に収まるように取る。η_c が大きいほど子は親の近くにできる
3. **突然変異**：成分ごとに確率 `mutation_rate`（既定 $1/d$）で、多項式突然変異 [8] により $y \leftarrow y + \delta_q (y^{\max} - y^{\min})$ とずらす。δ_q は分布指数 η_m（`eta_mutation`、既定 20）の分布から、範囲に収まるように取る
4. **世代交代**：親と子の $2N$ 個を非劣解ソートで順位づけし、順位の小さい順、同じ順位では混雑距離の大きい順に N 個を残す

収束と出力。 集団の値の平均 $\bar{\mathbf{f}}^n$ の、`windowsize_residual`（既定 20）世代前からの相対変化 $\|\bar{\mathbf{f}}^n - \bar{\mathbf{f}}^{n-W}\| / (\|\bar{\mathbf{f}}^0\| + 10^{-15})$ が `tolerance_residual`（既定 10^{-5}）以下で、初めて計算した値と同じでなければ止める（PSO と同じ守り）。`output_global.dat` には各世代の第 1 フロント（単目的では最良）を書く。エリート保存なので、単目的の最良は悪化しない。

並列化。 MPI では、選択・交叉・突然変異は rank 0 だけが行って全ランクに配り、子の評価だけを分担する（`allgather`）。ランクごとに乱数の種が違っても全ランクの集団が一致する。`num_parallel` によるスレッドの並列評価も PSO と同じ形で使える。

評価の回数。 初期集団の N 回と、1 世代に N 回である。

1.6 までの GA は、そのステップの最良との平均で交叉し全成分を同じ量だけずらすだけの試作で、`borealis.py` で外してあった。1.7.0 でこれを置き換えた。

4.4 ベイズ最適化

`src/Bayesian_optimization/Bayesian_optimization.py`。ガウス過程を代理モデルとし、獲得関数が最大になる点を次に評価する [6]。ガウス過程と獲得関数の計算は bayesian-optimization [10] に任せ、Borealis は候補を 1 つずつ受け取って評価し、値を登録する（suggest / register の形）。

1. 最初の `initial_design_numdata` 点は、範囲の一様乱数で評価する
2. そのあと `num_optiter` 回、これまでの全点に当てたガウス過程から獲得関数を作り、それが最大になる点を評価する

獲得関数は、期待改善量（EI）、改善確率（PI）、信頼限界（LCB。最小化では下限）から選ぶ。EI と PI の探索の度合い ξ は `acquisition_xi`、LCB の重み κ は `acquisition_weight` である。bayesian-optimization は最大化するので、最小化では値の符号を反転して登録する。評価の番号を目的関数に渡すこと、NaN・無限大を `value_nonfinite` に置き換えることは PSO・ABC と同じで、乱数の種は `optimizer.seed` で与える。1 回の評価のたびに代理モデルを当て直すので、外部コードの 1 回が重く、評価の回数を数十に抑えたいときに向く（節 8.6）。MPI での分担はしない。

1.6 までは GPyOpt [11] を使っていたが、保守が止まり numpy 2 で動かないので置き換えた（既知の問題 22）。設定の名前はそのまま使える。GPyOpt にしか無かった選択肢（ガウス過程以外のモデル、MCMC の獲得関数、獲得関数の図）は無くなった。

5 並列化

5.1 MPI による候補の分担

`mpirun -n P python3 borealis.py` で起動すると、PSO・ABC・GA は候補をランクに分ける。

- **PSO**：粒子を `np.array_split` で均等に連続に切り分ける（1.6 までは余りを最後のランクに載せていた。既知の問題 9)。各ランクは自分の粒子だけを評価し、ステップの終わりに位置・値・番号を rank 0 に集め（`gather`)、rank 0 がアーカイブとリーダーを作って全ランクに配る（`bcast`)。速度の更新は各ランクで自分の粒子について行う。履歴は最後に `allreduce` で合わせる
- **ABC**：雇用蜂は `np.array_split` で均等に分け、段の終わりに `Allreduce` で全ランクの食料源をそろえる。追従蜂は全食料源から選ぶので、自分の担当の外を更新しうる。そこで追従蜂の更新は `allgather` で集め、全ランクが同じ順序で適用する（担当の範囲だけを集めていた頃は、担当の外の更新が消えていた。既知の問題 25）
- **GA**：子を `np.array_split` で分けて評価し、値を `allgather` で集める。遺伝的な操作は rank 0 が行って `bcast` で配る（節 4.3）

ディレクトリの作成とファイルの書き出しは rank 0 だけが行う。外部コードが逐次のプログラムなら、この方式がそのまま効く。各ランクは 1 コアに固定され、そこから起動した外部コードもそのコアを継ぐので、P 個の外部コードが P 個のコアで並んで走る。

5.2 外部コードが MPI のとき：並列評価

外部コード自体が MPI で走る場合（いまは `shapeoptimizer` の SU2)、上の方式は使えない。

- MPI で起動したランクから、外部コードの `mpirun` を起動すると失敗する。子が親のジョブの環境変数（`OMPI_*`・`PMIX_*`）を継ぎ、自分を既存のジョブの 1 ランクと取り違えるためである（既知の問題 18）
- 起動できたとしても、子は親のランクの 1 コアの固定を継ぎ、すべての MPI のプロセスがその 1 コアに詰め込まれる
- **mpirun なしで起動しても**、mpi4py を読んだ時点（シングルトンの `MPI_Init`）で、これらの環境変数が C の側に足される。`subprocess` を `env=None` で呼ぶと子に継がれ、子の `mpirun` は何も出さずに終了コード 1 で終わる。Python の `os.environ` は起動したときの写しでこれらを含まないので、`env=dict(os.environ)` を明示的に渡すと起動する（既知の問題 19。Open MPI 4.1.6 で両方を確かめた）

そこで、Borealis は MPI なしの 1 プロセスで起動し、PSO が各ステップの粒子の評価をスレッドで同時に呼ぶ（`PSO.num_parallel`)。各スレッドは子プロセスの終了を待っているだけなので、Python のスレッドが同時に 1 つしか動けない制約（GIL）は効かない。結果は粒子の順に返るので、個人の最良とアーカイブの更新は逐次のときと同じ順で行い、PSO のアルゴリズムは変わらない。

アダプタの側は、評価ごとに「スロット」（コアの組）を 1 つ取り、テンプレートを写し、外部コードを `subprocess.Popen` で起動する。スロットのコアは、このプロセスに許されたコアの並び（`os.sched_getaffinity`。ジョブスケジューラの下ではジョブに割り当てられたコア）から順に取る。コアは環境変数 `BOREALIS_CPU_SET` で実行スクリプトに渡し、実行スクリプトはそれを `mpirun --cpu-set` に使う。独立に起動した複数の `mpirun` は互いを知らず、指定しなければどれも先頭のコアに固定するので、この指定が要る。スロットは外部コードの終了を待ってから返す。結果のファイルが出ないまま外部コードが終わったら、`penalty_failure` を目的関数の値として返し、最適化を続ける。スレッドから同時に呼ば

れるので、アダプタは os.chdir（プロセス全体に効く）を使わず、評価ごとの状態をインスタンスに持ってはならない。この条件を満たすアダプタだけが flag_parallel_evaluation = True を宣言し、宣言の無いアダプタで num_parallel > 1 を書くと止まる。オプティマイザは PSO・GA・ABC が、アダプタは shapeoptimizer・simple_function が対応している（既知の問題 17）。ABC は、雇用蜂の候補を段の始まりの食料源から作るので（逐次では先に更新された食料源を後の蜂が参照する）、並列のときは逐次と答えが変わる。初期化・追従蜂・偵察蜂の段は、候補が互いに依存しないので逐次と同じである。

測った速さは節 8.5 にある。

6 アダプタ

kind_adapter	中身
simple_function	解析的な目的関数（ずらした球関数 $f_k(\mathbf{x}) = \lvert\mathbf{x} - 2(k-1)\mathbf{1}\rvert^2$）。外部コードは要らない。検証に使う
example_externalcode	外部コードを呼ぶ雛形。曲線の当てはめ
tacode	軌道解析コード Tacode（Python 版と Fortran 版）。軌道の観測から大気の密度を推定する [12, 13]
heatcond	熱伝導の解析。表面の温度の履歴から熱流束を推定する
shapeoptimizer	SU2 と、preCICE で連成した形の変形（src_interface/）。抗力とモーメントの 2 目的
cage	cage の解析
user	利用者が自分のアダプタを置く場所

表 1　アダプタ。設定のキーは report/borealis_config_ja.pdf にある。

新しいアダプタは、雛形を写して 4 つのメソッド（__init__・initial_settings・reference_data_setting・objective_function）を書き、borealis.py の分岐に名前を足す。

時刻を比べるアダプタの注意。 tacode は、基準のデータの時刻ごとに計算の時刻の対応点を探し、前後の 2 点で線形に内挿して比べる。基準のデータの時刻の単位 reference.timeunit の day は**恒星日**（86164.0989 秒）である。1.6 までのこの探索には、基準のデータの時刻が計算の範囲の外にあると、Python の負の添字で配列の末尾の点と比べてしまう構造があった（既知の問題 6）。ISS の例では、基準のデータを暦日で作っていたためにこれが起き、時刻の合っていない 2 点を比べているのに、目的関数は 0.05〜0.06 というそれらしい値を返し続けた（既知の問題 23）。1.7.0 では範囲の外の点は比べず、その数を警告する。**目的関数の値が小さいことは、正しさの根拠にならない。** 最良の解の時系列を、基準のデータと図で比べること。

7 出力

結果は <手法>.result_dir に、Tecplot の ASCII の形で書く。

- filename_output（PSO・ABC・GA）：すべての候補の、各ステップの位置（PSO は速度も、GA は非劣解の順位も）、番号、誤差、残差。ステップ（世代）ごとに Tecplot のゾーンを 1 つ書く
- filename_global（PSO・ABC・GA）：各ステップのアーカイブ（PSO）、それまでの最良（ABC）、第 1 フロント（GA。単目的では最良）の番号・値・位置。PSO の多目的では、最終のステップの行が最終のアーカイブで、**全履歴の Pareto フロントではない**（既知の問題 10）
- filename_output（ベイズ最適化）：評価した順の点・値・番号

番号は、PSO では $nN + i + 1$（n は 0 から、i は 0 から）で、アダプタのケースのディレクトリの番号と一致する。src_helper/ の道具（最良の解の取り出し、最良でないケースの削除、履歴のアニメーション）は、

この番号でケースを引く。

8 検証

検証には、解析解の分かっている `simple_function` を使う。5 次元で、範囲は各成分 $[-10, 10]$ である。ケースは `report/verification/` にあり、乱数の種を `optimizer.seed` で固定してある。`report/run_verification.py` がまとめて走らせる。どのケースも 1 秒ほどで終わる。

8.1 単目的：球関数

$f(\mathbf{x}) = |\mathbf{x}|^2$ で、最小は $\mathbf{x} = \mathbf{0}$ の $f = 0$ である。PSO は 20 粒子、$w = 0.5$、$c_1 = c_2 = 1.0$、ABC は雇用蜂・追従蜂とも 20 匹、`visit_limit` $= 10$、GA は 20 個体で、交叉・突然変異は既定の値とした。

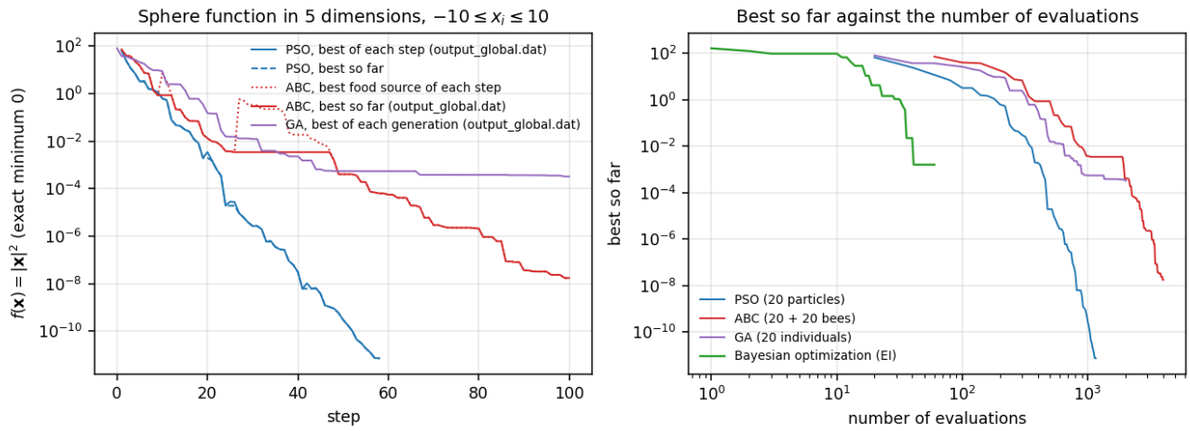


図 1　球関数の収束。左：ステップに対して。PSO の実線は各ステップのアーカイブ（そのステップの最良）、破線はそれまでの最良。ABC の実線は `output_global.dat` の値（それまでの最良）、点線は各ステップの食料源の中の最良（`output_ABC.dat`）。GA は各世代の最良（エリート保存なので、それまでの最良と同じ）。右：評価の回数に対するそれまでの最良（ベイズ最適化を含む。ABC の評価の回数は偵察蜂の分を数えていない）。

図 1 に結果を示す。

- PSO は 58 ステップ（1160 回の評価）で収束の判定が成り立ち、$f = 7.3 \times 10^{-12}$ で止まった。ステップごとの最良はほぼ単調に減り、それまでの最良とほぼ重なった。リーダーがそのステップの最良であることは、この問題では収束を妨げていない
- ABC は 100 ステップ（約 4000 回の評価）を回りきり、それまでの最良は $f = 1.7 \times 10^{-8}$ だった。各ステップの食料源の中の最良（点線）は、ステップ 10 と 27 で 1 桁以上悪化している。偵察蜂が、改善の止まった最良の食料源を捨てて作り直したためである。それまでの最良（実線、`output_global.dat`）は単調に下がる（1.6 まではファイルに点線の値を書いていた。既知の問題 16）
- GA は 100 世代（2020 回の評価）で $f = 3.2 \times 10^{-4}$ だった。最良は単調に下がるが、50 世代目（5.5×10^{-4}）からはほとんど進まない。実数値の GA は、最適解の近くで精度を詰めるのが PSO より遅い。精度が要るなら `eta_mutation`・`eta_crossover` を大きくする（子を親の近くに作る）か、PSO を使う

8.2 2 目的：Pareto フロント

$f_1 = |\mathbf{x}|^2$、$f_2 = |\mathbf{x} - \mathbf{c}|^2$、$\mathbf{c} = 2 \cdot \mathbf{1}$ とする。三角不等式から、任意の $\mathbf{x}$ で

$$\sqrt{f_1} + \sqrt{f_2} = |\mathbf{x}| + |\mathbf{x} - \mathbf{c}| \;\geq\; |\mathbf{c}| = 2\sqrt{5} \tag{8}$$

が成り立ち、等号は $\mathbf{x}$ が $\mathbf{0}$ と $\mathbf{c}$ を結ぶ線分の上にあるときに限る。この線分が Pareto 集合で、Pareto フロントは $\sqrt{f_1} + \sqrt{f_2} = 2\sqrt{5}$ である。そこで、アーカイブの点の外れを $\delta = |\mathbf{x}| + |\mathbf{x} - \mathbf{c}| - 2\sqrt{5}$ (≥ 0) で測る。PSO は 40 粒子で、100 ステップを回した（収束の判定のしきい値を十分小さくした）。

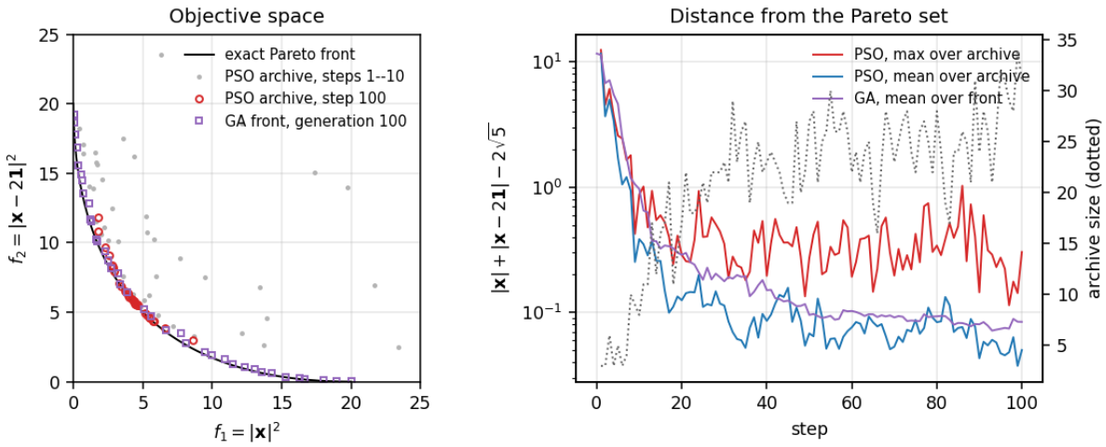


図 2　2 目的の PSO と GA。左：目的の空間での PSO のアーカイブ、GA の最終世代の第 1 フロント、厳密な Pareto フロント。右：Pareto 集合からの外れ δ（PSO はアーカイブの最大と平均、GA はフロントの平均）と、PSO のアーカイブの大きさ（点線）。

図 2 に結果を示す。最終のアーカイブは 31 点で、δ の平均は 0.050、最大は 0.30 だった。δ の平均は、最初のステップの 11.7 から 20 ステップ目の 0.13 まで 2 桁下がり、そこからは 0.04〜0.20 のあいだで揺れて、それ以上は下がらない。アーカイブはそのステップの粒子だけから作るので、前のステップのよりよい点が残らないこと、リーダーを一様に乱択するので群れがフロントの一部（最終のアーカイブでは f_1 が 1.8〜8.6）に集まることが、この頭打ちの理由と見ている。フロントの全体を細かく得たいなら、全ステップの `output_PSO.dat` から非劣解を選び直すのがよい（既知の問題 10）。

同じ問題を GA（40 個体、100 世代。NSGA-II の世代交代）で解くと、最終世代の第 1 フロントは 40 点すべてで、f_1 が 0.05〜20.0 と**フロントの両端まで均等に覆った**（図 2 の四角）。δ の平均は 0.084、最大は 0.21 で、PSO より平均はわずかに大きいが最大は小さい。δ の平均は、20 世代目に 0.29、60 世代目に 0.10 と、最後まで下がり続ける（PSO のように頭打ちにならない）。GA は親と子を合わせて残すので、よい点が世代をまたいで残り、混雑距離で選ぶので点がフロントに広がる。多目的では、フロントの全体を得たいなら GA、とくに f_1 の小さい側・大きい側の端が要るときは GA を使うのがよい。

8.3　再現性

同じ種で 2 回走らせると、PSO・ABC・GA の 5 つのケースの `output_global.dat` はすべて 1 ビットも違わずに一致した。逐次の実行では、Borealis の答えは種だけで決まる。PSO と GA の並列評価（`num_parallel`）でも、乱数はスレッドの外で引くので、逐次と同じ答えになる。ABC の並列評価は雇用蜂の段だけ逐次と答えが変わるが、並列の数にはよらない（どちらもテストで確かめている）。MPI で走らせると、各ランクの乱数の列が別になり、プロセス数を変えたときの再現性は無い。

8.4 テストと CI

`tests/` に pytest のテストがあり、`python3 -m pytest tests` で走る（手元で 42 件、約 27 秒。ベイズ最適化の 3 件は bayesian-optimization が無ければ飛ばす）。中身は次のとおりである。

- `orbital` の道具：時刻の単位、既定のパス、対応点の探索（以前のループと同じ点を返すことの照合と、範囲の外の扱い）、測地座標の往復、目的関数の値の検査、外部コードの起動（終了ステータス、古い結果のファイル、MPI の環境変数が子に漏れないこと）
- 設定の検査：知らないキー・古い名前の警告と、同梱のすべての `borealis.yml` に知らないキーが無いこと
- 手法を通しで：PSO の収束と再現性、並列評価が逐次と同じ答えになること、最大化の符号、2 目的の Pareto 集合への接近、ABC のそれまでの最良が単調なこと、多目的を拒むこと、GA の交叉と突然変異が範囲に収まること・非劣解ソート・最良の単調性・再現性・並列評価・2 目的でフロントを覆うこと、ベイズ最適化の収束・再現性・最大化・使えないモデルを拒むこと
- MPI：PSO・ABC・GA を 13 個の候補・3 プロセスで（`mpirun` がある環境だけ）
- `shapeoptimizer`：MPI を使わないダミーの外部コードで、並列評価・単目的と 2 目的・コアの割り当て・失敗のときのペナルティ

1.6 の版のソースに同じテストを当てると、18 件のうち 16 件が失敗する（直した不具合をテストが捕まえることの確認）。GitHub Actions（`.github/workflows/ci.yml`）が、push と pull request のたびに Python 3.9 と 3.12 で、構文の検査・テスト・このレポートの検証ケースを走らせる。

8.5 並列評価の速さ

節 5.2 の並列評価を、AMD Ryzen 9 9900X（物理 12 コア、論理 24）と Open MPI 4.1.6 で測った。外部コードは 2 つ用意した。1 つは、`mpirun -np 2 --cpu-set` で起動されて 2 秒眠り、各ランクのコアの固定（`sched_getaffinity`）を書き出すダミーの MPI のプログラムである。もう 1 つは、`testcase/work_shapeoptimizer/work/work_cylinder2D_multiobj` の SU2 + preCICE を 6 粒子・2 ステップに縮めたものである。どちらも `num_parallel` = 3、1 件 2 コアとした。

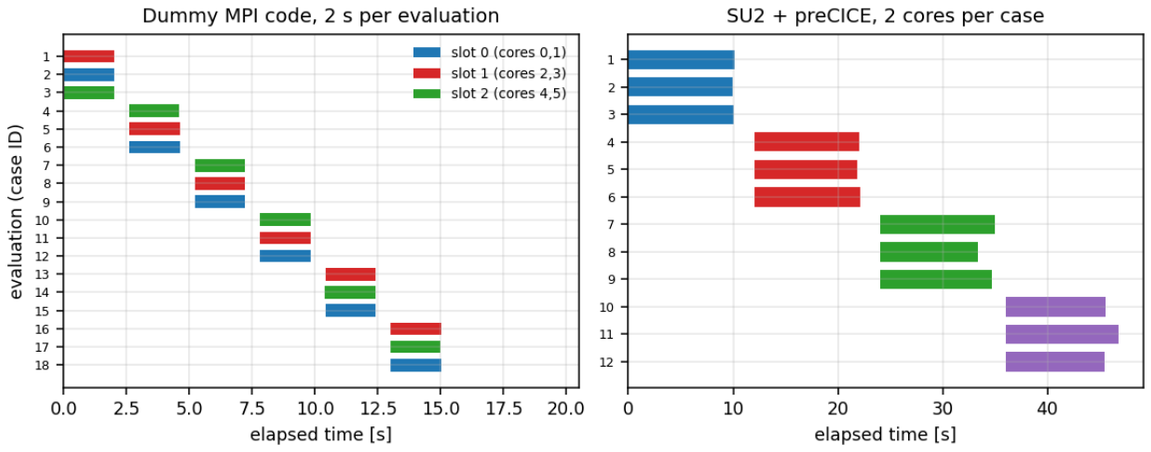


図 3　並列評価のタイムライン。左：ダミーの MPI のプログラム（色はスロット）。右：SU2 + preCICE（色はステップの組）。

結果を図 3 と表 2 に示す。

外部コード	評価	1 件 [s]	全体 [s]	1 件ずつなら [s]	速さ
ダミーの MPI のプログラム	18	2.00	15.0	36.0	2.4 倍
SU2 + preCICE	12	9.3〜10.9	46.8	120.7	2.6 倍

表 2　並列評価の速さ（`num_parallel` = 3）。「1 件ずつなら」は各評価の時間の和。

- 3 件ずつ同時に走り、各ケースのランクは割り当てたコアの組（0,1 / 2,3 / 4,5。SMT の兄弟を含む）に固定されていた。重なりは無かった
- 理想の 3 倍に届かないのは、組と組のあいだの隙間による。ダミーでは 0.6 秒（結果のファイルを見る間隔 `polling_interval` は 0.2 秒）、SU2 では約 2 秒（既定の 3 秒）だった。PSO は 1 ステップの全粒子がそろうまで待つので、組の中でいちばん遅いケースが組の長さを決める。**1 件が短いときは `polling_interval` を縮める**
- ダミーを常に失敗させると、6 件すべてが `penalty_failure` を返し、止まらずに最適化を終えた

スパコンのジョブ環境での確認はまだである（既知の問題 20）。ジョブに割り当てられたコアが 0 番から始まらないと、絶対番号の `--cpu-set` は外れる。

8.6　ベイズ最適化

同じ 5 次元の球関数を、ベイズ最適化（EI、$\xi = 0.01$、初期の 10 点 + 50 回）で解いた（`report/verification/bo_sphere`）。図 1 の右に、評価の回数に対するそれまでの最良を、PSO・ABC と並べて示す。

- ベイズ最適化は 42 回目の評価で $f = 1.6 \times 10^{-3}$ に達し、そのあとの 18 回では改善しなかった
- 同じ値に達するまでに、PSO は 420 回（21 ステップ）、GA は 840 回（41 世代）、ABC は約 1940 回（48 ステップ）の評価を要した
- 一方、それ以上の精度（10^{-6} より下）には、ベイズ最適化は 60 回では届かない。ガウス過程の当てはめと獲得関数の最大化に、60 回の評価で合わせて 12 秒（開発機）かかった

外部コードの 1 回が重く、評価の回数を数十に抑えたいときはベイズ最適化、評価が安く精度が要るときは PSO、という使い分けになる。同じ種で 2 回走らせると、評価した点と値はすべて一致した。

9　適用の例：ISS の軌道からの大気密度の推定

`testcase/work_tacode_fortran` に、ISS の軌道から高度 410〜440 km の大気密度を推定する例が 3 つある。基準の軌道は、Celestrak の TLE を SGP4 で伝播し、間欠的な GNSS の疑似観測からガウス過程回帰で作り直したものである。Tacode で軌道を計算し、高度の差が小さくなるように密度の補正を PSO（100 粒子・50 ステップ）で探す。補正の与え方が 3 つで違う。

3 つとも軌道は同じ程度に合い、F10.7 の方式は 1 つの物理的に解釈できる量（F10.7A $\approx$ 128）で同じ合い方を出した。ただし、基準の軌道は SGP4 による計算で実測の密度ではないので、得られた補正は「Tacode の軌道のモデルと SGP4 の差」を吸収している面が強い。物理的な大気密度としてそのまま読むべきではない。

この例の途中で、目的関数を黙って誤らせる問題が 2 つ見つかった。1 つは節 6 の恒星日と暦日の取り違え（既知の問題 23）、もう 1 つは Tacode の大気のテーブルが ISS の高度を覆っておらず、密度の補正が実は効いていなかったこと（既知の問題 24）である。どちらも、目的関数の値ではなく、最良の解の軌道と密度の分布を図で見て見つかった。

ケース	補正の与え方	収束	目的関数	高度の RMS 残差
`work_tacode_iss`	高度の区分ごとの係数（4 区分、階段関数）	–	0.0008	0.06 km
`work_tacode_iss_pchip`	同じ制御点を PCHIP で滑らかにつないだ係数	57 ステップ	0.00095	0.071 km
`work_tacode_iss_f107`	太陽の電波の強さ F10.7 を 1 つの量として最適化し、NRLMSISE-00 で密度を作り直す	42 ステップ	0.00089	0.067 km

表 3　ISS の例の結果。数値は 2026 年 8 月の実行の記録（`CHANGELOG.md`）から引いた。このレポートのために計算し直したものではない。

10　既知の問題

一覧と詳細は、リポジトリの外に置いた開発の記録にある。1.7.0 で、GA を実装し、ベイズ最適化を bayesian-optimization に置き換え（22）、答えを黙って誤らせるもの（最大化の符号、NaN、`hour` の係数、時刻の対応点の回り込み、外部コードの失敗、既定のパス、単目的の `shapeoptimizer`）を直し、テストで押さえた。いま残っているのは次のものである。

- 知らないキーは警告だけで、止まらない（4）
- ABC の収束の判定は、改善の無いステップで成立しうる（5）
- PSO のアーカイブとリーダーはそのステップの粒子からしか選ばず、位置も速度も制限しない（10、21）
- ペナルティが違反の量によらない定数（13）
- 並列評価はベイズ最適化に無く、アダプタは一部だけ（17）、スパコンでのコアの割り当ては未確認（20）
- `tacode` の `weight_time` が誤差に入らない（27）

参考文献

[1] J. Kennedy and R. Eberhart, “Particle swarm optimization,” *Proc. ICNN’95*, vol. 4, pp. 1942–1948, 1995.

[2] Y. Shi and R. Eberhart, “A modified particle swarm optimizer,” *Proc. IEEE ICEC*, pp. 69–73, 1998.

[3] C. A. Coello Coello, G. T. Pulido, and M. S. Lechuga, “Handling multiple objectives with particle swarm optimization,” *IEEE Trans. Evol. Comput.*, vol. 8, no. 3, pp. 256–279, 2004.

[4] D. Karaboga, “An idea based on honey bee swarm for numerical optimization,” Technical Report TR06, Erciyes University, 2005.

[5] D. Karaboga and B. Basturk, “A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm,” *J. Global Optim.*, vol. 39, pp. 459–471, 2007.

[6] D. R. Jones, M. Schonlau, and W. J. Welch, “Efficient global optimization of expensive black-box functions,” *J. Global Optim.*, vol. 13, pp. 455–492, 1998.

[7] K. Deb and R. B. Agrawal, “Simulated binary crossover for continuous search space,” *Complex Systems*, vol. 9, pp. 115–148, 1995.

[8] K. Deb and M. Goyal, “A combined genetic adaptive search (GeneAS) for engineering design,” *Computer Science and Informatics*, vol. 26, no. 4, pp. 30–45, 1996.

[9] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algo-

rithm: NSGA-II,” *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, 2002.

[10] F. Nogueira and contributors, “Bayesian Optimization: Open source constrained global optimization tool for Python,” `https://github.com/bayesian-optimization/BayesianOptimization`, version 3.4.0, 2026.

[11] The GPyOpt authors, “GPyOpt: A Bayesian optimization framework in Python,” `http://github.com/SheffieldML/GPyOpt`, 2016.

[12] Y. Takahashi, M. Saito, N. Oshima, and K. Yamada, “Trajectory reconstruction for nanosatellite in very low Earth orbit using machine learning,” *Acta Astronautica*, vol. 194, pp. 301–308, 2022.

[13] T. Sakai and Y. Takahashi, “Atmospheric density estimation in very low Earth orbit based on nanosatellite measurement data using machine learning,” *Aerospace Science and Technology*, vol. 153, 109418, 2024.